

The FADE Emulator: A Flexible & Anisotropic Department-of-Experts Model for Rapid Posterior-Predictive Estimation

JFG

24 March 2026

Changelog

Rather than making a new post with the changes I have made, I will keep a changelog of new updates to both the writeup and the model here.

23/03/26

- (i) Added some notes about accelerating the inference speed

18/03/26

- (i) Added a changelog, and backdated some entries
- (ii) Shifted notation to allow emulator to predict quantities in arbitrary dimensions, $y \rightarrow \mathbf{y}$.
- (iii) Added a reweighting protocol that means changing the latent priors requires retraining the emulator, not re-running the mechanistic model
- (iv) Generated some priors for the emulator parameters

17/03/26

- (i) Wrote the autodiff section & optimisation of the MAP acquisition
- (ii) Renamed the model (again) to the FADE & rewrote the underlying analogy to ‘department’ based. Commensurate notation change.

Note: only the 3 most recent logs will be rendered – the rest are hidden by the compiler.

1 THE MOTIVATION

There exists many decades of literature on ways to estimate a mapping $\hat{M} : \mathcal{P} \rightarrow \mathcal{O}$ between some parameter space $\mathcal{P}$ to an observable space $\mathcal{O}$, conditioned on a set of observed data $\mathbf{T} = \{(\mathbf{y}, \mathbf{x})\}$. The vast majority of this work is of ‘estimators’, wherein one is able to reliably compute $\mathbb{E}(\hat{\mathbf{y}}|\hat{\mathbf{x}}, \mathbf{T})$ for some new proposed set of covariates $\hat{\mathbf{x}} \notin \mathbf{T}$.

For our purposes however, we are more interested in a *predictor*, which would enable us to predict not just the behaviour of the mean, but the entire distribution, $p(\hat{\mathbf{y}}|\hat{\mathbf{x}}, \mathbf{T})$.

One reason to prefer a basic Gaussian Process Emulation

(And equivalently, a BLUP) is that it - by construction - is simultaneously a mean-emulator and a density-emulator. That means that you automatically generate a distribution (a Gaussian one) along with the estimated value.

However this is predicated wholly on the *a priori* assumption that the process is Gaussian in nature - and thus that it can solely be defined by its first two moments. This works perfectly for the mean function (where GPs are universal approximators), but if we have any reason to disbelieve the Gaussian nature (and we probably do, in general), then the resulting distribution is merely a convenient first-order approximation.

In short: Gaussian Processes are insufficient when attempting to reconstruct an accurate *distribution*, rather than just the *mean*.

2 THE DATA & THE MECHANISTIC MODEL

We suppose that there exists some observable $\mathbf{y} \in \mathbb{R}^q$ which may be extracted from a function $\hat{M}$ evaluated at a parameter vector $\mathbf{P} \in \mathbf{\Pi}$:

$$\mathbf{y} = \hat{M}(\mathbf{P}) \quad (1)$$

For the cases which are of interest to us, $\hat{M}$ is a Semi-Mechanistic Model which is very expensive to compute - on the order of minutes to hours for each $\mathbf{P}$.

‘Semi-mechanistic’ Models

We term the models of interest to us as ‘semi-mechanistic’ because we permit the model to be stochastic, and therefore $\mathbf{y}$ to be a realisation of an underlying random variable

$$\mathbf{y} \sim Y$$

Despite this, the model is a ‘principled construction’: derived from an axiomatic model of how the underlying reality functions.

2.1 Parameter Promotion

The full parameter space $\mathbf{\Pi}$ may in general be very high dimensional. For the purposes of statistical emulation we divide it into two subspaces:

(i) **Conditional Variables** These are the covariates of interest for the final emulator. They define the *emulation space*

$$\mathcal{E} = \mathbb{R}^n \quad (2)$$

and are denoted by $\mathbf{x}$.

(ii) **Latent Variables** The remaining parameters of the mechanistic model. These variables are not explicitly represented in the emulator and instead contribute stochastic variability to the output distribution. They are denoted $\mathbf{z}$, and we assume that there exists some prior on their values, which may be conditioned on the covariates $P(\mathbf{z}|\mathbf{x})$.

Why use manual promotion?

PCA and related nonlinear projection methods provide a method in which the machine computes a projection into the space of the ‘most important variables’:

$$\hat{\mathbf{x}} = \mathcal{T}(\mathbf{P}) \quad (3)$$

It would then be possible to learn $p(\mathbf{y}|\hat{\mathbf{x}})$, which is an easier task since $\hat{\mathbf{x}}$ is a lower-dimensional entity, and the projection has (in theory) captured the bulk of the underlying correlations.

However, this approach requires specifying a full parameter vector $\mathbf{P}$ whenever the model is used to generate a prediction. For complex mechanistic systems this is often undesirable. For example, in the mosquito model we wish to predict outcomes conditioned on the gene drive parameters, without needing to specify rainfall realisations and mosquito lifetimes.

In short, we wish to simultaneously marginalise over these parameters, and fit the covariates of interest.

2.2 Generating Data

The ‘obvious’ solution for generating the data we need for our emulator would be to sample $\mathbf{x}$ uniformly from $\mathcal{E}$, and $\mathbf{z}$ from the latent prior, $P(\mathbf{z})$, and then compute $\mathbf{y} = \hat{M}(\mathbf{x}, \mathbf{z})$. Our dataset would then be $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^D$.

However, this would have the major drawback that if – at a later date – our priors on the latent variables were updated¹, then we would need to completely regenerate the training data. This would be a significant bottleneck in computation time: for the models of interest to us, the vast majority of the computational expense will be in creating $\mathcal{D}$.

We therefore generate our on $\mathbf{z}$ sample using a more generous distribution function (or even a uniform one), $P_{\text{sample}}(\mathbf{z})$ and generate *two* datasets:

$$\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i, \pi_i)\}_{i=1}^D \quad \mathcal{Z} = \{\mathbf{z}_i, \zeta_i\}_{i=1}^D \quad (4)$$

The ordering between these sets is the same (so $\mathbf{y}_i =$

$\hat{M}(\mathbf{x}_i, \mathbf{z}_i)$ is true). The parameters π_i and ζ_i are defined as:

$$\begin{aligned} \pi_i &= P_{\text{sample}}(\mathbf{z}_i) \\ \zeta_i &= \frac{P_{\text{latent}}(\mathbf{z}_i)}{\pi_i} \end{aligned} \quad (5)$$

Where P_{sample} is the distribution used to generate $\{\mathbf{z}\}$ and P_{latent} is the *current prior on the latent variables*. This means that $\mathcal{D}$ is immutable (unless more data is added), and changes to the prior require simply recomputing ζ_i – a trivial task.

When computing the log-posterior conditioned on the observed evidence, we simply need to correct by a factor ζ_i :

$$\mathcal{L}(\mathcal{D}, \mathcal{Z}) = \sum_{i=1}^D \zeta_i \times \ln p(\mathbf{y}_i | \mathbf{x}_i, \dots) \quad (6)$$

We can see that if $\zeta_i = 1$ (which occurs when $P_{\text{sample}} = P_{\text{latent}}$), then this reduces to the obvious case. Otherwise, it means that we have an effective sample size

$$D^* = \frac{(\sum_i \zeta_i)^2}{\sum_i \zeta_i^2} \quad (7)$$

The choice of sampling distribution must therefore be informed on a best-guess understanding of how the latent prior may evolve to ensure that D^* is a reasonable size for both the current latent prior, and any future updates. If $D^* \ll D$, then it will be necessary to generate more data, which is to be avoided under the current paradigm.

3 DERIVING THE MODEL

3.1 Model Ingredients

We now review several model classes that provide useful components for constructing a probabilistic emulator.

3.1.1 Partition Models

Partition models divide the covariate space into a collection of disjoint regions, each associated with a separate statistical model. The simplest example is a regular grid in $\mathcal{E}$ where each bin contains a histogram or kernel density estimate of the observed outputs.

A more flexible construction uses a Voronoi tessellation. Given a set of seeds $\{\mathbf{s}_i\}_{i=1}^{N_d} \subset \mathcal{E}$, each point $\mathbf{x}$ is assigned to the nearest seed according to a distance metric:

$$\text{cell}(\mathbf{x}) = \arg \min_i \left\{ \sqrt{(\mathbf{x} - \mathbf{s}_i)^\top G_i (\mathbf{x} - \mathbf{s}_i)} \right\}. \quad (8)$$

Here G_i is a positive-definite metric tensor associated with the i^{th} cell. In many practical implementations (such as the [paper we have been following](#)) this metric is restricted to a diagonal form

$$G_i = \text{diag}(\mathbf{w}_i) \quad (9)$$

with $\sum_j [w_i]_j = 1$, which preserves the usual geometric properties of a Voronoi tessellation such as linear cell boundaries and simply connected regions.

¹ Let’s say that we got a better prediction of the rainfall/aestivation triggers/population density etc.

Partition models are attractive because they allow different statistical behaviour in different regions of the parameter space. However, they suffer from a major limitation: predictions are typically discontinuous across cell boundaries. Capturing smoothly varying behaviour therefore requires a large number of partitions.

3.1.2 Mixture of Experts

An alternative approach is the *mixture of experts* model, in which the conditional distribution is represented as a weighted sum of simpler component models:

$$p(\mathbf{y}|\mathbf{x}) = \sum_{i=1}^{N_e} w_i(\mathbf{x}) p_i(\mathbf{y}|\mathbf{x}), \quad (10)$$

where p_i are expert distributions and the weights satisfy

$$\sum_i w_i(\mathbf{x}) = 1. \quad (11)$$

In many machine learning applications the weights are generated by a gating network, for example

$$w_i(\mathbf{x}) = \frac{\exp(\mathbf{k}_i^T \mathbf{x} + b_i)}{\sum_j \exp(\mathbf{k}_j^T \mathbf{x} + b_j)}. \quad (12)$$

More complex architectures allow the covariates $\mathbf{x}$ to first be transformed through multiple hidden layers before the mixture weights are computed.

Mixture models provide smooth interpolation between experts, but the gating mechanism is often purely algebraic and does not explicitly exploit the geometric structure of the covariate space. In short, it is a black box that ‘works’ with little explainability as to why one expert was more heavily weighted than the other.

Terminology & Analogy

This is termed a ‘mixture of experts’ because it is helpful to think of the process as a set of nodes which are making predictions as to the answer, and then reaching a consensus based on how close the query point is to each of their ‘areas of expertise’: the closer to an expert the point is, the more weight their voice is given to the final consensus.

3.1.3 Kernel-Based Expert Interpolation

A simpler and more geometrically interpretable weighting scheme can be constructed using kernels. Suppose each expert is associated with a location $\mathbf{c}_i$ in the covariate space. We define weights using a radial basis function kernel:

$$w_i(\mathbf{x}) = \frac{K(d(\mathbf{x}, \mathbf{c}_i | G))}{\sum_\ell K(d(\mathbf{x}, \mathbf{c}_\ell | G))}. \quad (13)$$

Here

$$d(\mathbf{x}, \mathbf{y} | G) = \sqrt{(\mathbf{x} - \mathbf{y})^T G (\mathbf{x} - \mathbf{y})} \quad (14)$$

is a Mahalanobis distance induced by the metric G , and $K(d)$ is a radial kernel satisfying $K(0) = 1$ and decaying with increasing distance.

Under this construction the prediction at $\mathbf{x}$ is obtained by smoothly interpolating between nearby experts. If $\mathbf{x}$ lies close to $\mathbf{c}_k$ then $w_k(\mathbf{x}) \approx 1$ and the prediction is dominated by the k^{th} expert.

3.2 Department-Expert Models

A Department-Expert Model combines the geometric structure of partition models with the smooth interpolation properties of kernel mixtures.

We assume that the model contains

- a set of Voronoi tessellation seeds $\{\mathbf{s}_k\}_{k=1}^{N_d}$, and
- a set of experts $\{\mathbf{c}_i, p_i\}_{i=1}^{N_e}$ located within the covariate space.

If the partitioning were applied in a *hard* manner, predictions would only depend on experts located in the same Voronoi cell as the query point, giving the following form:

$$p(\mathbf{y}|\mathbf{x}) = \sum_{k=1}^{N_d} T_k(\mathbf{x}) \sum_{i=1}^{N_e} w_i^k(\mathbf{x}) p_i(\mathbf{y}), \quad (15)$$

$$w_i^k(\mathbf{x}) = \frac{T_k(\mathbf{c}_i) K(d(\mathbf{x}, \mathbf{c}_i))}{\sum_j T_k(\mathbf{c}_j) K(d(\mathbf{x}, \mathbf{c}_j))}, \quad (16)$$

$$T_k(\mathbf{a}) = \delta_{k, \text{cell}(\mathbf{a})} = \begin{cases} 1 & \text{cell}(\mathbf{a}) = k \\ 0 & \text{else} \end{cases} \quad (17)$$

This is identical to the pure mixture model, but with T_k removing elements from the sum if the points do not belong to k - and since the only non-zero contribution to the outer sum is from the k to which $\mathbf{x}$ belongs, it ensures that $p(\mathbf{y}|\mathbf{x})$ only gets contributions from experts in the same Voronoi cell.

Terminology & Analogy

To continue the analogy of ‘experts’ reaching a consensus, we term each of the cells generated by this partition as a *department*. Each point in the covariate space – and each expert – is then assigned a ‘department’, and only the experts in that department may contribute to the consensus.

This is a *Strict Department of Expert* (SDE) model.

However, SDE models suffer from discontinuities in the likelihood whenever $\mathbf{x}$ crosses a cell boundary. These discontinuities make the likelihood surface non-differentiable and therefore unsuitable for gradient-based optimisation.

To avoid this difficulty we introduce a *flexible department* border. Rather than T_k acting to assigning each point to a single cell, we define it as a probability that a point belongs to the department:

$$T_k(\mathbf{x}) = P(\text{cell}(\mathbf{x}) = k | \Theta) = \frac{K_2(d(\mathbf{x}, \mathbf{s}_k))}{\sum_j K_2(d(\mathbf{x}, \mathbf{s}_j))}. \quad (18)$$

Here K_2 is the ‘department kernel’ a distinct radial basis function that we choose *a priori*. When the kernel K_2 is sharply peaked the model approaches an SDE, while broader

kernels produce smoother transitions between regions of parameter space. This defines a *Flexible Department of Experts* (FDE) model.

Terminology & Analogy

In the analogy: experts are no longer purely confined to one department, but rather spend varying amounts of time visiting each department. The more time they spend in one department (corresponding to a higher T_k), the more weight their voice is given in a consensus.

3.3 A Flexible & Anisotropic Department of Experts

In the basic FDE model formulation above, we assumed that the metric G was constant across space, and thus that d was similarly constant.

The final ingredient of our model – the Flexible & Anisotropic Department of Experts (FADE) Model – is to equip each department with a learned metric. Each seed $\mathbf{s}_k$ is associated with a metric tensor G_k , and the distance between two covariate vectors is evaluated locally as

$$d_k(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top G_k (\mathbf{x} - \mathbf{y})}. \quad (19)$$

If G_k is restricted to a diagonal form (as in Payne et al.) this simply rescales the coordinate axes, assigning different relative importance to each covariate within the k^{th} region.

Relaxing this restriction allows G_k to contain off-diagonal terms, which encode correlations between covariates and therefore rotate and stretch the local coordinate system. Consequently each department does not merely partition the parameter space but instead defines a local *correlation topology*.

Terminology & Analogy

To finalise the analogy of a FADE model - the varying metric means that each department is associated with a unique way of interpreting the covariate space and the correlations within it.

In this analogy, each department has a unique interpretation of the same technical terms - the word ‘stress’ might mean something different to a physicist (“force per unit area”), a biologist (a physiological response to a stimuli) and a linguist (the prominence given to a syllable in a word). When an expert is ‘working in’ a department, they must use the terminology of that department: an expert close to the border of two departments will rapidly switch between terminologies – whilst those ensconced in their own department will use only their own jargon.

The full specification of a FADE model is detailed in table 1

3.4 Parameter Priors

As good Bayesian statisticians, we should want to place priors which genuinely reflect our prior knowledge and expectations of the system on the parameters of the model. Some of these

priors are expressed explicitly through our choice of distribution function and Kernels - for example if, in the univariate case ($\mathbf{y} = y$), we know that $y \geq 0$ (or rather, $p(y < 0) = 0$), then our choice of $\mathcal{F}$ should be a distribution with support $y \in [0, \infty)$: this will ensure that our prior condition is automatically met.

Need to discuss the rest of these priors!

- N_s, N_d

In general, we would like our model to follow an Occam-type principle: to be as parsimonious with explanations as possible. The marginalisation process inherent in the posterior predictive *should* act to prevent overfitting (as the volume of the posterior space grows super-exponentially, whilst the volume of a well-fitting posterior remains constant). However, we may wish to help things along by imposing an additional prior:

$$P(N_i) = \mathcal{N}_i \begin{cases} N_i^{-\gamma} & 1 \leq N_i \leq N_i^{\max} \\ 0 & \text{else} \end{cases}$$

Where γ and $N_i^{\max}$ are chosen a priori. This penalises models which use an excess of experts and departments.

- $\mathbf{c}_i, \mathbf{s}_k$

These are the spatial positions of the experts and departments. In general our priors here will be uniform and bounded on some domain:

$$P(\mathbf{c}_i) = \begin{cases} \frac{1}{|\mathcal{V}|} & \mathbf{c}_i \in \mathcal{V} \\ 0 & \text{else} \end{cases}$$

And similarly for $\mathbf{s}_i$. The domain $\mathcal{V}$ is chosen based on domain knowledge - in the case of our mosquito model (ω, η), it is clear that $\mathcal{V}$ is the Cartesian unit square, $0 \leq c_j^i, s_j^i \leq 1$.

We might also wish to place a prior which encourages the model to populate departments with at least one expert (assuming that) within the boundaries of the cell. The flexible model does not require this, but in the limit where the departments become *strict*, a department without any experts is meaningless.

We should, however, be careful that our priors retain the permutation invariance of the model. If this invariance is broken, we would need to reconsider the degeneracy of the Laplace approximation (see below).

A reasonable prior might be that we disbelieve any model where there exists a department where all $T_k(\mathbf{c}_i) \ll 1$. This corresponds to a prior:

$$P(\Theta) = \left(\prod_{k=1}^{N_d} T_k^{\max} \right)^\alpha \quad (26)$$

$$T_k^{\max} = \max_i (\{T_k(\mathbf{c}_i)\})$$

In short, this quantity finds the ‘closest’ expert (as per the metric of the department), and finds out their contribution to the department. If this contribution is low, the department is penalised.

- $\{\phi_{ij}^k\}$

These are the ‘Cholesky-parameters’ which define our metric. In the absence of any data, we might reasonably expect the prior to be Euclidean in nature, which is equivalent to saying that our prior obeys $\mathbb{E}(\phi_{ij}^k) = 0$. A suitable prior might then be the standard isotropic multivariate normal:

$$\phi \sim \mathcal{N}(\mathbf{0}, \sigma^2 I) \quad (27)$$

$$p^{\text{fade}}(y|\mathbf{x}, \Delta, \Theta, \eta) = \sum_{k=1}^{N_d} T_k(\mathbf{x}) \sum_{i=1}^{N_e} w_i^k(\mathbf{x}) p_i(\mathbf{y}), \quad (20)$$

$$w_i^k(\mathbf{x}) = \frac{T_k(\mathbf{c}_i) K_1(d_k(\mathbf{x}, \mathbf{c}_i))}{\sum_{j=1}^{N_e} T_k(\mathbf{c}_j) K_1(d_k(\mathbf{x}, \mathbf{c}_j))}, \quad (21)$$

$$T_k(\mathbf{x}) = \frac{K_2(d_k(\mathbf{x}, \mathbf{s}_k))}{\sum_{j=1}^{N_d} K_2(d_j(\mathbf{x}, \mathbf{s}_j))}, \quad (22)$$

$$d_k(\mathbf{x}, \mathbf{y}) = \sqrt{(\mathbf{x} - \mathbf{y})^\top G_k (\mathbf{x} - \mathbf{y})} \quad (23)$$

Where:

- (i) $\mathbf{y} \in \mathcal{Y} \subset \mathbb{R}^q$ is the value to be emulated. The space $\mathcal{Y}$ is constrained to be a subset of $\mathbb{R}^q$ by the choice of $\mathcal{F}$ and the prior.
- (ii) $\mathbf{x} \in \mathcal{E} \subset \mathbb{R}^n$ is the covariate input vector
- (iii) $\Delta = (N_d, N_e)$ are our discrete model parameters
 - (a) N_d is the number of Voronoi seeds used (due to the anisotropic metric, the space may be split into $n \geq N_d$ disjoint regions)
 - (b) N_e is the number of experts populating the space
- (iv) $\eta = (K_1, K_2, \mathcal{F})$ are the model hyperparameters - our final result will be conditioned on these values.
 - (a) $K_1(d)$ is the *expert kernel*, an isotropic distance kernel of some kind, used to determine how quickly the information is exchanged between experts in the same department
 - (b) $K_2(d)$ is the *department kernel*, an isotropic distance kernel used to determine how easily experts can communicate between departments
 - (c) $\mathcal{F}$ is a family of probability distribution functions that each expert may choose from.

- (v) $\Theta = \left(\left\{ \mathbf{s}_k, \{\phi_{ij}^k\} \right\}_{k=1}^{N_p}, \{\mathbf{c}_i, \boldsymbol{\xi}_i\}_{i=1}^{N_e} \right)$ specifies our partitioned-expert scheme

- (a) $\mathbf{s}_k \in \mathcal{E}$ are the coordinates seeds of the k^{th} Voronoi pseudo-partition
- (b) $\{\phi_{ij}^k\}_{0 \leq j \leq i \leq n}$ are a set of parameters ($n(n+1)/2$ of them) which are used to define a lower triangular matrix with positive diagonal elements:

$$[L_k]_{ij} = \begin{cases} \exp(\phi_{ij}^k) & i = j \\ \phi_{ij}^k & i > j \\ 0 & \text{else} \end{cases} \quad (24)$$

This then defines a unique metric, $G_k(\{\phi^k\}) = L_k L_k^\top$, which is guaranteed to be positive definite and symmetric. ϕ_{ij}^k is otherwise unconstrained

- (c) $\mathbf{c}_i \in \mathbb{R}^n$ are the coordinates of the i^{th} expert.
- (d) $\boldsymbol{\xi}_i$ is the parameter set which specifies p_i in the family $\mathcal{F}$:

$$p_i(\mathbf{y}) = \mathcal{F}(y; \boldsymbol{\xi}_i) \quad (25)$$

In the case where $\mathbf{y} = y \in \mathbb{R}$ and $\mathcal{F}$ is a univariate Gaussian, $\boldsymbol{\xi}_i = (\mu_i, \sigma_i)$.

Table 1. The full specification of a FADE model.

• $\boldsymbol{\xi}_i$ These are the parameters of the expert model, $p_i(y) = \mathcal{F}(y; \boldsymbol{\xi}_i)$. The prior on these depends on $\mathcal{F}$ and our expectations of the model. An unimaginative prior would be to assume some 'standard' $\hat{\boldsymbol{\xi}}$, and then set

$$P(\boldsymbol{\xi}) = \mathcal{N}(\boldsymbol{\xi} | \hat{\boldsymbol{\xi}}, \sigma^2 I) \quad (28)$$

$P(\Theta, \Delta)$ and our training data $\mathcal{D} = (\mathbf{x}_j, y_j)_{j=1}^D$, then the log-posterior² takes the form:

$$\mathcal{L}(\Theta, \mathcal{D}, \mathbf{h}, \Delta) = \sum_{j=1}^D \ln(p(y_j | \mathbf{x}_j, K_1, K_2, \mathcal{F}, \Theta)) + \ln P(\Theta, \Delta) \quad (29)$$

However, for the purposes of an emulator, we wish to be able to generate a new prediction, conditioned on the data that we have already observed. This is the posterior-predictive distri-

4 THE EMULATOR

Assuming that $p^{\text{fade}}(y|\mathbf{x}, \Delta, \Theta, \mathbf{h})$ is a suitable approximation of the conditional posterior on y , and that we have a prior

² The log of the conditional probability of observing $\mathbf{T}$, given our prior $\mathbf{P}$

bution, and is computed from:

$$p(\hat{y}|\mathbf{x}, \mathcal{D}, \mathbf{h}) = \sum_{\Delta} \int p(y|\mathbf{x}, \Theta, \Delta, \mathbf{h}) \exp(\mathcal{L}(\Theta, \mathcal{D}, \mathbf{h})) d\Theta \quad (30)$$

The issue is that this sum-of-integrals is completely intractable to compute. The standard method of computing it would be to use MCMC methods to draw values from the posterior, and use that to construct a Monte-Carlo approximation to the integral. However, with $\mathcal{L}$ a trans-dimensional quantity (as N_e, N_d changes the dimensionality of the input space varies), this too becomes intractable.

We may, however, make use of the Laplace Approximation, which is directly equivalent to expanding the log-posterior to the first non-zero order around the MAP value:

$$\mathcal{L}(\Theta, \mathcal{D}, \mathbf{h}, \Delta) \approx \mathcal{L}(\hat{\Theta}, \mathcal{D}, \mathbf{h}, \Delta) + \frac{1}{2} (\Theta - \hat{\Theta})^\top \frac{\partial^2 \mathcal{L}}{\partial \Theta^2} (\Theta - \hat{\Theta}) \quad (+ \approx 0) \quad (31)$$

Where $\hat{\Theta}$ is the MAP solution, the parameter set which maximises the value of $\mathcal{L}$. Therefore:

$$p(\Theta|\mathcal{D}, \mathbf{h}, \Delta) \approx \exp(\mathcal{L}(\hat{\Theta})) \times \mathcal{N}(\Theta|\hat{\Theta}, H^{-1}) \quad (32)$$

Where H is the (negative) Hessian of the log-posterior:

$$H = -\nabla_{\Theta}^2 \mathcal{L} \quad (33)$$

Degeneracy

The Laplace approximation relies on the assumption that the probability is dominated by a single mode, which decays sufficiently quickly away from the maximum that the vast majority of information is captured by the first order expansion. As $D \rightarrow \infty$, this will become more and more true: **with one vital caveat.**

Because both our seeds and our experts are unconstrained, the model is invariant under a permutation of $\mathbf{c}_i$ and $\mathbf{s}_j$ (so long as the associated parameters ξ_i and metrics G_j are similarly permuted). Our posterior is therefore highly multi-modal, with peaks at each permutation which lie far apart in parameter space - not captured by the simple Laplace approximation.

However, each of these permutations results in an identical model $p(y|\mathbf{x}, \Theta, \Delta, \mathbf{h})$, so as long as the prior is similarly permutation-invariant, it suffices to simply multiply our posterior by this degeneracy factor:

$$p(\Theta|\mathcal{D}, \mathbf{h}, \Delta) \approx g(\Delta) \exp(\mathcal{L}(\hat{\Theta})) \times \mathcal{N}(\Theta|\hat{\Theta}, H^{-1}) \quad (34)$$

$$g(\Delta) = N_d! N_e! \quad (35)$$

With this quantity in hand, it is then possible to do one of the following:

- (i) Draw samples from this multivariate normal quantity (a trivial task) to construct a Monte-Carlo integral
- (ii) Follow the example of INLA, and carefully choose a

‘handful’³ of quadrature points, and use these to compute the integral.

Assuming the first choice is made, this reduces to an S -element sum over some set of target points $\mathcal{S} = \{\Theta^{(1)}, \Theta^{(2)}, \dots\} \sim \mathcal{N}(\Theta|\hat{\Theta}, H^{-1})$ to approximate the integral:

$$\begin{aligned} p(\hat{y}|\hat{\mathbf{x}}, \mathcal{D}, \mathbf{h}, \Delta) &\propto \sum_{\Delta} g(\Delta) \sum_{s \in \mathcal{S}} p^{\text{fade}}(\hat{y}|\hat{\mathbf{x}}, \Theta^{(s)}, \mathbf{h}, \Delta) \\ &= \sum_{N_d=1}^{U_s} \sum_{N_e=1}^{U_e} \sum_{s \in \mathcal{S}(\Delta)} \sum_{k=1}^{N_d} \sum_{i=1}^{N_e} g(N_d, N_e) T_k(\mathbf{x}|\Theta^{(s)}) w_i^k(\mathbf{x}|\Theta^{(s)}) p_i(y|\Theta^{(s)}) \end{aligned} \quad (36)$$

Where U_p and U_e are the upper bounds of the number of partitions and experts permitted into the model (these are incorporated into $\mathbf{h}$).

Inference of a new position $\hat{\mathbf{x}}$ requires computing a sum of N Gaussians, where

$$N \approx \frac{U_p(U_p + 1)U_e(U_e + 1)}{4} \times S \quad (37)$$

4.1 Accelerating Inference

Although this sum may be quicker to compute than the original model $\hat{M}$, it is not necessarily *quick* - especially when U_e or U_p are large. It is therefore useful to compute if there are any ways to speed this process up.

4.1.1 Block-Correlation

If one assumes that the correlation structure takes the form of a block matrix of the following form:

$$H_{\Theta} = H_{\text{expert}} \oplus H_{\text{other}} = \begin{pmatrix} H_{\text{expert}} & \mathbf{0} \\ \mathbf{0} & H_{\text{other}} \end{pmatrix} \quad (38)$$

It is then possible to precompute:

$$\tilde{p}_i(y|N_d, N_e) = \frac{1}{S} \sum_{s \in \mathcal{S}} p_i(y|\text{expert}^{(s)}, N_d, N_e) \quad (39)$$

At inference time, one then simply needs to compute the reduced sum:

$$p(\hat{y}|\mathbf{x}) = \sum_{N_d=1}^{U_s} \sum_{N_e=1}^{U_e} \sum_{i=1}^{N_e} \tilde{p}_i(y) \left(\sum_{s \in \mathcal{S}(\Delta)} \sum_{k=1}^{N_d} W_{ik}(\mathbf{x}) \right) \quad (40)$$

This is a computational saving because when computing $p(y)$ we typically compute it on a grid of y values of resolution R .

Precomputing the mean-expert opinion means that at inference time we can avoid computing $N_d R \times S$ sums of the ‘opinions’, an instead merely compute $S N_d$ sums of the weights and then one R -element multiplication.

³ Exact quantity TBD

4.1.2 Truncating Sums

The calculation can also be accelerated by noting that the sum over N_d, N_e does not need to be fully computed: using the same logic as behind the Laplace approximation: if there exists an optimal pair $\Delta = (N_d, N_e) = (\nu_d, \nu_e)$ which maximises the value of $\exp(\mathcal{L}(\hat{\Theta}(\Delta)))g(\Delta)$, then the summation will be dominated by the nearby terms:

$$\sum_{N_d=1}^{U_e} \sum_{N_e=1}^{U_e} \approx \sum_{N_d=\nu_d-\delta_d}^{\nu_d+\delta_d} \sum_{N_e=\nu_e-\delta_e}^{\nu_e+\delta_e} \quad (41)$$

We can also generalise this and allow $\nu_e = \nu_e(d)$.

We can also accelerate the computing of the individual W_{ik} values by noting that the values of $T_k(\mathbf{x})$ and $T_k(\mathbf{c}_i)$ tend to zero quickly, and will most likely be dominated by terms $0.1 \lesssim T_k < 1$ ⁴.

If $T_k(x) \ll 1$, therefore we may omit the computation of $\sum_i w_{ik} p_i$, on the assumption that this term is subdominant.

In short, we can easily estimate which departments we can ignore for a prediction value $\mathbf{x}$, so the sum over $i = 0 \rightarrow N_e$ can be reduced to $i \in K(\mathbf{x})$ where $\dim(K(\mathbf{x})) \ll N_d$.

Similarly, we can compute which *experts* need to be computed based on $T_k(\mathbf{c}_i)$ and w_{ik} . We may then generate a subset of non-vanishing expert contributions $E_k(\mathbf{x})$, where $\dim(E_k(\mathbf{x})) \ll N_e$.

Both of these statements are, naturally, dependent on the actual value of Θ being used - and since this value is varied during the sum - the suitable cutoff values are non-trivial. However, since this evaluation only needs to occur once (for caching), it is suffice to compute S MAP-draws and find an good-enough cutoff.

If we compute and cache values of $(\nu_e, \delta_e, \nu_d, \delta_d, \{K(\mathbf{x})\}, \{E_k(\mathbf{x})\})$ at train-time, then the inference simplifies down to a sum of N' Gaussians, where

$$\langle N' \rangle = 4\delta_d\delta_e \langle \dim(K)\dim(E_k) \rangle \times S \quad (42)$$

IF we assume that, on average, each summation truncation removes a fraction $\bar{\alpha}$ of elements to be summed, then:

$$\frac{N'}{N} = (1 - \bar{\alpha})^4 \quad (43)$$

$\bar{\alpha}$ need only be ~ 0.15 for this to result in a 2x speedup of inference, and if it reaches 0.5 (which is not a ludicrous assumption - a *lot* of terms will be zero!), then we get a 20x speedup.

5 THE SPLAT-EXPERTS

In the previous example, we assumed that the ‘experts’ contributed a single, simple PDF. The rest of the model complexity was regarding how to mix these expert opinions together into a consensus which could be evaluated.

⁴ The proof of this statement is a bit handwavey. We’re basically appealing to the idea that the odds of there being a position which is equidistant between >10 experts is infinitesimally small. This gets less true as the dimensionality of the covariate space grows.

If, however, the covariate space included spatial coordinates⁵, then this can drastically inhibit the model, as it is going to have to capture the full spatial complexity of the model, as well as the variations as a function of the other input parameters. Prediction is similarly delayed: to make an N -dimensional prediction would require N^2 queries into the posterior predictive. The emulator is quick - but there are limits!

We can get around this by using a trick inherited from the computer graphics industry. Rather than trying to think of a spatial grid as a series of correlated ‘pixels’ on a grid, onto which we have to make a full suite of predictions, we can instead think aggregating that correlated information into a Gaussian-pixel (‘gaussel’).

To do this, we again separate our parameter space, this time into three components:

(i) **Spatial Coordinates** The coordinates in a final p -dimensional space, each position of which is to be equipped with a PDF. Denoted as $\mathbf{z} \in \mathbb{R}^p$.

(ii) **Conditional Variables** The remaining non-spatial parameters, $\mathbf{x}$

(iii) **Latent Variables** The remaining parameters of the mechanistic model.

In the previous model, each expert (at a position $\mathbf{x} = \mathbf{c}_i$) was equipped with a single PDF. In this model, we equip them with N_m gaussels, each of which has a weight, α_j , a scale height ρ_j , a mean $\boldsymbol{\mu}_j \in \mathbb{R}^p$ and a variance $\boldsymbol{\Sigma}_j \in \mathbb{R}^{p \times p}$.

Instead of using $\mathcal{F}$, we assume that we have some exponential-family function h (and canonical link function g), the expert can be understood to be making a prediction:

$$p_i = \sum_j^{N_m} \alpha_j h(y | g^{-1}(\rho_j \mathcal{N}(\mathbf{z} | \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j))) \quad (44)$$

Each expert is therefore makes a prediction of a latent field, ℓ_j :

The Simple Approach

This quantity p_i could be passed into the standard FADE formulation, requiring almost no modifications. However, in order to build up a map of predictions across $\mathbf{z}$, you would need to evaluate the posterior predictive at every point on the mesh. This remains costly - the only advantage you have really gained is in prompting the optimiser as to your expected covariate structure.

However, instead of interpolating across the latent field, we instead interpolate individually across the components, coming to a consensus **on the parameters of the splat field**:

⁵ In our case, if $\mathbf{x} = (\text{lat}, \text{long}, \alpha, \eta)$, where α is the dispersal rate and η the fitness cost

$$\begin{pmatrix} \alpha_j^{\text{con}}(\mathbf{x}) \\ \rho_j^{\text{con}}(\mathbf{x}) \\ \mu_j^{\text{con}}(\mathbf{x}) \\ \Sigma_j^{\text{con}}(\mathbf{x}) \end{pmatrix} = \sum_{k=1}^{N_d} \sum_{i=1}^{N_e} T_k(\mathbf{x}) w_i^k(\mathbf{x}) \begin{pmatrix} \alpha_j^i \\ \rho_j^i \\ \mu_j^i \\ \Sigma_j^i \end{pmatrix} \quad (45)$$

The consensus splat fields then have the form:

$$\ell_j^{\text{con}}(\mathbf{z}, \mathbf{x}) = \rho_j^{\text{con}}(\mathbf{x}) \mathcal{N}(\mathbf{z} | \mu_j^{\text{con}}(\mathbf{x}), \Sigma_j^{\text{con}}(\mathbf{x})) \quad (46)$$

With the emulated PDF being:

$$p(y | \mathbf{x}, \mathbf{z}, \Theta, \Delta, \mathbf{h}) = \sum_j \alpha_j^{\text{con}}(\mathbf{x}) h(y | \ell_j^{\text{con}}(\mathbf{z}, \mathbf{x})) \quad (47)$$

6 THE LEARNING PHASE

In order to efficiently compute the posterior predictive, we need to be able to draw samples from the posterior distribution - which we have made simple via the Laplace Approximation, which assumes it to be a multivariate normal around the MAP position. The ‘learning’ phase therefore amounts to computing and caching the values of $\hat{\Theta}$ and Σ^{-1} .

$$\hat{\Theta}(\Delta) = \arg \max_{\Theta} \left(\sum_{j=1}^D \ln \left(p^{\text{lapse}}(y_j | \mathbf{x}_j, \Theta, \mathbf{h}, \Delta) \right) + \ln P(\Theta, \Delta) \right) \quad (48)$$

$$\Sigma^{-1}(\Delta) = -\nabla_{\Theta}^2 \left(\sum_{j=1}^D \ln \left(p^{\text{lapse}}(y_j | \mathbf{x}_j, \Theta, \mathbf{h}, \Delta) \right) + \ln P(\Theta, \Delta) \right) \quad (49)$$

We note that these quantities are not only a function of $\Delta = (N_e, N_p)$, but the *dimensionality* of is a function of these values too: the more partitions and experts we include, the more parameters are needed to describe them.

Computing the MAP value in such a case is best done with a Stochastic Gradient Descent Method, which in turn means that we need to be able to evaluate the gradient:

$$\frac{\partial \mathcal{L}}{\partial \Theta} = \left(\sum_j \frac{1}{p(y_j | \mathbf{x}_j, \Theta, \mathbf{h}, \Delta)} \frac{\partial p(y_j | \mathbf{x}_j, \Theta, \mathbf{h}, \Delta)}{\partial \Theta} \right) + \frac{\partial P(\Theta, \Delta)}{\partial \Theta} \quad (50)$$

For truly optimal performance, it would behave us to analytically compute these gradients. However this is complex, fiddly and time consuming. It is probably more efficient to make use of an automatic differentiation framework.

6.1 An Introduction to Automatic Differentiation

Automatic Differentiation is a technique which allows a computer to automatically find the partial derivative of a function f with respect to one of its inputs, without requiring the user to encode the derivative of the object. This assumes the function f is a composite of ‘standard’ functions.

Consider the *dual numbers* which consists of the ordered pairs $\langle x, y \rangle \in \mathbb{R} \times \mathbb{R}$. The basic arithmetic of these numbers obeys:

$$\begin{aligned} \alpha + \beta \langle x, y \rangle &= \langle \alpha + \beta x, \beta y \rangle \\ \langle x, y \rangle \pm \langle w, v \rangle &= \langle x \pm w, y \pm v \rangle \\ \langle x, y \rangle \cdot \langle w, v \rangle &= \langle xw, xv + yw \rangle \\ \frac{\langle x, y \rangle}{\langle w, v \rangle} &= \left\langle \frac{u}{v}, \frac{yw - xv}{w^2} \right\rangle \end{aligned}$$

This can be recognised as equivalent to the *chain rule*. An equivalent formulation to the ordered pairs is the introduction of the infinitesimal ϵ which is $\epsilon > 0$, but $\epsilon^2 = 0$ - i.e. it is the “smallest non-zero number”:

$$x + \epsilon y \longleftrightarrow \langle x, y \rangle \quad (51)$$

Careful consideration show that common functions $F : \mathbb{R} \rightarrow \mathbb{R}$ can be generalised to $F : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R} \times \mathbb{R}$

$$\sin(\langle x, y \rangle) = \langle \sin(x), y \cos(x) \rangle$$

$$\ln(\langle x, y \rangle) = \left\langle \ln(x), \frac{y}{x} \right\rangle$$

$$\exp(\langle x, y \rangle) = \exp(x) \langle 1, y \rangle$$

$$(\langle x, y \rangle)^{\langle u, v \rangle} = \langle x^u, y u x^{u-1} + v \ln(x) x^u \rangle$$

In short, if $f(x)$ is a function on $\mathbb{R}$, then the equivalent function on the duals is defined as:

$$f(\langle x, y \rangle) = \left\langle f(x), y \frac{df}{dx} \right\rangle \quad (52)$$

This means that if f is some composite of functions for which the dual equivalents are known, then $\tilde{f}$ can be constructed through the compositing of those dual equivalents. That is, if $a(x)$ and $b(x)$ are functions for which the dual equivalents $\tilde{a}$ and $\tilde{b}$ are known, then:

$$c(x) = b(a(x)) \longleftrightarrow \tilde{c}(\langle x, y \rangle) = \tilde{b}(\tilde{a}(\langle x, y \rangle)) \quad (53)$$

This means that we may then trivially find the derivative of $c(x)$, by computing $\tilde{c}(\langle x, 1 \rangle)$:

$$\tilde{c}(\langle x, 1 \rangle) = \tilde{b}(\langle a(x), a'(x) \rangle) \quad (54)$$

$$= \langle b(a(x)), a'(x) b'(a(x)) \rangle \quad (55)$$

The ‘automatic’ part is that we naturally compute this quantity automatically, as a natural part of the dual arithmetic. Therefore, by defining a library of dual-equivalent functions, it is possible to then automatically compute the derivatives of arbitrary composites of this library.

6.2 Multivariate Duals

Things are slightly more complex in the case of a multivariate function, $y = f(x_1, x_2, \dots, x_n)$ - though not by much. In this case, it can be shown that:

$$f(\langle x_1, y_1 \rangle, \langle x_2, y_2 \rangle, \dots) = \left\langle f(x_1, x_2, \dots), \frac{\partial f}{\partial \mathbf{x}} \cdot \mathbf{y} \right\rangle \quad (56)$$

Where $\mathbf{x} = (x_1, x_2, \dots)$ and $\mathbf{y} = (y_1, y_2, \dots)$. This directional derivative is not particularly useful for us: but if we choose $\mathbf{y} = \hat{e}_j$ (i.e. $y_i = \delta_{ij}$), then it simplifies to:

$$f(\langle x_1, 0 \rangle, \langle x_2, 0 \rangle, \dots, \langle x_j, 1 \rangle, \dots) = \left\langle f(x_1, x_2, \dots), \frac{\partial f}{\partial x_j} \right\rangle$$

(57)

That is, if the duals are equal to scalars except for one element (which is equal to 1), then the result is the *partial derivative with respect to the parameter associated with the non-scalar dual*.

We may therefore construct the full gradient vector by computing this quantity each time, extracting the second element, and storing it in an array.

Similarities & Efficiency

This is broadly similar to constructing the vector by the method of finite differences, in which we approximate:

$$\frac{\partial f}{\partial x_j} = \frac{f(\mathbf{x} + \delta \hat{e}_j) - f(\mathbf{x})}{\delta} \quad (58)$$

However, this method is prone to numerical instability. If δ is too large, it's a bad approximation. If δ is too small, the result can be highly numerically unstable, and not representative of the true value.

Automatic differentiation, however, is significantly more robust against these: there is no requirement for 'small things to cancel out'. There may still be some floating point error accumulation for deeply nested functions - but it does not require thinking about a valid choice of δ .

Computing the gradient of a function on $\mathbb{R}^n$ therefore requires n evaluations of the dual-function - each one setting a different dual equal to 1 (whilst zeroing the others). This might be significantly less efficient than a cleverly designed analytical gradient, which might take advantage of re-using quantities in multiple components of the gradients. Cacheing such precomputable elements becomes harder in the case of automatic differentiation - but not impossible. With a suitable choice of the ordering of the input values, one can ensure that some values can be pre-computed and stored - and only require recomputing when the associated parameter is perturbed.

6.2.1 Scalar-Vector Duals

In the above example of multivariate-duals, we extended the notion to an array of ordered pairs. Whilst this worked, the result was a *directional* derivative, $g(\mathbf{v}) = \mathbf{v} \cdot \nabla f$. Since there is no division operator on $\mathbb{R}^N$, in order to extract the full gradient⁶ we have to iterate the computation N times, where $N = \dim(\mathbf{x})$, with $\mathbf{v}_i = \hat{e}_i$.

This is potentially problematic if f is costly to compute. Consider the toy model where $f(\mathbf{x}) = \exp(\hat{e}_0 \cdot \mathbf{x})$, where $\dim(x) = N \gg 1$. An analytical approach would tell us that:

$$(\nabla f) \cdot \hat{e}_i = \begin{cases} f & i = 0 \\ 0 & \text{else} \end{cases} \quad (59)$$

We could therefore compute the function value f and the

⁶ which is what we are more interested in for the purposes of gradient descent: we wish to know the direction to perturb our query. We don't even really care about the magnitude of the gradient in the naive case where the learning rate is fixed.

gradient $\mathbf{g}$ with a single computation of $\exp$, and then N if-calls and memory assignments. The autodiff approach would require N evaluations of $\exp$, and N ifs and memory assignments. Given that computing $\exp$ is an order of magnitude slower than if/memory assignment, we can see why this might be problematic.

Instead, we consider the space of scalar-vector duals, $\mathcal{D}_N = \mathbb{R} \times \mathbb{R}^N$:

$$\mathcal{X} \in \mathcal{D}_N = \langle x, \mathbf{x}' \rangle \quad (60)$$

These obey the following arithmetic:

$$\begin{aligned} \mathcal{X} + \alpha \mathcal{Y} &= \langle x + \alpha y, \mathbf{x}' + \alpha \mathbf{y}' \rangle \\ \mathcal{X} \cdot \mathcal{Y} &= \langle xy, y\mathbf{x}' + x\mathbf{y}' \rangle \\ \exp(\mathcal{X}) &= \exp(x) \langle 1, \mathbf{x}' \rangle \end{aligned} \quad (61)$$

From which it follows that

$$\begin{aligned} g(\mathcal{X}) &= \left\langle g(x), \frac{dg}{dx} \mathbf{x}' \right\rangle \\ h(\mathcal{X}, \mathcal{Y}) &= \left\langle h(x, y), \frac{\partial h}{\partial x} \mathbf{x}' + \frac{\partial h}{\partial y} \mathbf{y}' \right\rangle \end{aligned} \quad (62)$$

Therefore:

$$f(\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_n) = \left\langle f(x_1, x_2, \dots, x_n), \sum_{i=1}^n \frac{\partial f}{\partial x_i} \mathbf{x}'_i \right\rangle \quad (63)$$

Therefore if $\mathbf{x}'_i = \hat{e}_i$:

$$f(\mathcal{X}_1, \mathcal{X}_2, \dots, \mathcal{X}_n) = \langle f(x_1, x_2, \dots, x_n), \nabla f \rangle \quad (64)$$

This construction allows us to compute ∇f directly - at the additional cost of each dual now requiring a vector $\mathbb{R}^n$, instead of a single scalar.

Speeding Up the Vectors

There are various approaches that we might take to accelerating these computations. If we assume that the vast majority of operations occur on an 'unmixed' portion of parameter space, then the vector becomes sparse - only a few elements are non-zero at any given moment. It is therefore quicker to not keep an entire vector, but a map of non-zero entries. This is slower than a vector when the map density approaches the density of a vector.

Alternatively, we could just rely on simple hardware acceleration for vectorisation: if we ensure that the vector is memory-aligned then the loop over the vector can take advantage of (blockwise) SIMD parallelisation. We can explore which of these works best.

6.2.2 Scalar-Vector-Matrix Duals

Our 'learning' phase is the computing and caching of the Laplace approximation of the posterior. This requires finding not just $\hat{\Theta}$, but the deviation around it, $\Sigma = -\nabla^2 f(\hat{\Theta})$. It would be eminently possible therefore to estimate the Laplacian by a method of finite differences, but given we have gone to the trouble of generating this lovely automatic differentiation framework, it would be a shame to then make the Laplacian a poor approximation.

We also note that we only need to compute the Hessian once

per cache-loop: the gradient needs to be optimised to Hell and back, but the Hessian can accept a slightly poorer scaling behaviour.

We consider the *hyperdual* space, $\mathcal{H}_N = \mathcal{D}_N \times \mathbb{R}^{N \times N}$ associated with the dual space $\mathcal{D}_N$. This takes the form of an ordered tuple of a scalar, a vector $\mathbb{R}^N$ and a matrix $\mathbb{R}^{N \times N}$:

$$\mathcal{X} = \langle x, \mathbf{x}', X'' \rangle \quad (65)$$

This obeys the arithmetic:

$$\begin{aligned} \mathcal{X} \pm \alpha \mathcal{Y} &= \langle x \pm \alpha y, \mathbf{x}' \pm \alpha \mathbf{y}', X'' \pm \alpha Y'' \rangle \\ \mathcal{X} \cdot \mathcal{Y} &= \langle xy, y\mathbf{x}' + x\mathbf{y}', yX'' + xY'' + \mathbf{x}'(\mathbf{y}')^\top + \mathbf{y}'(\mathbf{x}')^\top \rangle \end{aligned} \quad (66)$$

Where $\mathbf{ab}^\top$ is the outer product of the vectors.

$$\begin{aligned} g(\mathcal{X}) &= \langle g(x), g'(x)\mathbf{x}', g'(x)X'' + g''(x)(\mathbf{x}'(\mathbf{x}')^\top) \rangle \\ h(\mathcal{X}, \mathcal{Y}) &= \left\langle h(x, y), \frac{\partial h}{\partial x}\mathbf{x}' + \frac{\partial h}{\partial y}\mathbf{y}', \frac{\partial h}{\partial x}X'' + \frac{\partial h}{\partial y}Y'' + \frac{\partial^2 h}{\partial x^2}(\mathbf{x}'\mathbf{x}'^\top) \right\rangle \end{aligned} \quad (67)$$

If $X'' = Y'' = 0$, and $\mathbf{x}' = \hat{e}_x, \mathbf{y}' = \hat{e}_y$, then it is clear that:

$$\begin{aligned} g(\langle x, \hat{e}_x, \mathbf{0} \rangle) &= \langle g(x), g'(x)\hat{e}_x, g''(x)\delta_{xx} \rangle \\ h(\langle x, \hat{e}_x, \mathbf{0} \rangle, \langle y, \hat{e}_y, \mathbf{0} \rangle) &= \left\langle h(x, y), \frac{\partial h}{\partial x}\hat{e}_x \frac{\partial h}{\partial y} + \hat{e}_y, \sum_{i,j \in \{x,y\}} \frac{\partial^2 h}{\partial i \partial j} \delta_{ij} \right\rangle \end{aligned} \quad (68)$$

Where $\delta_{ij} = \hat{e}_i \hat{e}_j^\top$ is the matrix which is zero everywhere except at i, j , where it is 1. Therefore, in the general multi-variate case:

$$\begin{aligned} f(\mathcal{X}) &= \left\langle f(\mathbf{x}), \sum_i \frac{\partial f}{\partial x_i} \hat{e}_i, \sum_{ij} \frac{\partial^2 f}{\partial x_i \partial x_j} \delta_{ij} \right\rangle \\ &= \langle f(\mathbf{x}), \nabla f, \nabla^2 f \rangle \quad \text{when } [\mathcal{X}]_i = \langle x_i, \hat{e}_i, \mathbf{0} \rangle \end{aligned} \quad (69)$$

As before, this then lets us compute the Hessian with a single trivial pass through the function – at the cost of storing the matrix in the intermediary values.

6.2.3 Slapdash Notation

I have been somewhat careless with my notation – the functions $f(\mathcal{X} \in \mathcal{H}_N)$, $f(\mathcal{X} \in \mathcal{D}_N)$ and $f(\mathbf{x} \in \mathbb{R}^N)$ are fundamentally different objects: each of them operates on an entirely different arithmetic.

However, I have been careless precisely because we *can* be careless in C++ by using template parameters:

```
class Dual
{
    Dual(int N) : Dimension(n), Gradient(
        std::vector<double>(n,0)), Value(0)
    {};
    int Dimension;
    double Value;
    std::vector<double> Gradient;
}
//define operator overloads to function as
dual arithmetic
Dual operator+(Dual a, Dual b){...}
```

```
Dual operator*(Dual a, Dual b)
{
    assert(a.Dimension == b.Dimension);
    Dual out(a.Dimension)
    out.Value = a.Value * b.Value
    for (int i = 0; i < out.Dimension; ++i)
    {
        out.Gradient[i] = a.Value * b.
            Gradient[i] + b.Value * a.
            Gradient[i];
    }
}
//define common composable functions
Dual sin(Dual x)
{
    double c = cos(x.Value); // only
        compute cos once - then re-use N
        times
    auto & g = x.Gradient;
    Dual output;
    output.Value = sin(x.Value);
    std::transform(g.begin(), g.end(), output.
        Gradient.value(),
        [&](double g_i){return c * g_i});
}
// And do the same for Hyperdual, defining
arithmetic and composable functions

template<class T>
T lossFunction(double dataValue, std::vector
    <T> & param)
{
    T output;
    int N = input.size();
    // A test example assuming an N/3
        dimensional gaussian mixture
    for (int i = 0; i < N; i+=3)
    {
        auto mu = param[3*i];
        auto sigma = param[3*i + 1];
        auto w = param[3*i + 2];
        auto d = (dataValue - mu)/sigma;
        output += w * exp(-d*d);
    }
    return output;
}

int main(int argc, char**argv)
{
    double d = //some value;
    double x_scalar;
    Dual x_dual;
    Hyperdual x_hyper;

    double y_scalar = lossFunction(d,
        x_scalar)
    Dual y_dual = lossFunction(d, x_dual)
    Hyperdual y_hyper = lossFunction(d,
        hyper);
}
```

I.e., so long as the loss function is written as a templatable function of objects which obey the cor-

rect arithmetic, and which have suitable compositing libraries written (i.e. a definition for `exp(Dual x)` and `atan2(Hyperdual x, Hyperdual y)`), then the compiler takes care of everything for us, and we have a highly optimisable function which can be written as if it is a simple arithmetic expression on the scalars.